Comparison of Floating-Point Representations' Efficacy in Polynomial Regression

John Baldwin
Johns Hopkins University
jbaldw27@jhu.edu

I. Introduction

In modern computing systems, machine learning (ML) algorithms and related workloads are becoming increasingly common and more integrated from the highest levels of software to the lowest levels of hardware. As ML becomes more prevalent, it is important to consider all aspects of the subject to find potential improvements and optimizations. Here, I consider one of the most fundamental optimizations: the floating-point representation.

Floating-point numbers are the basic operand of many, if not all, ML algorithms which leaves them open as a key place for optimization to occur. In this project, four different representations were considered: IEEE-754 Double Precision, IEEE-754 Single Precision, IEEE-754 Half Precision, and the newer Brain Float 16 (bfloat16). The bit fields of each representation can be seen in Figure 1.

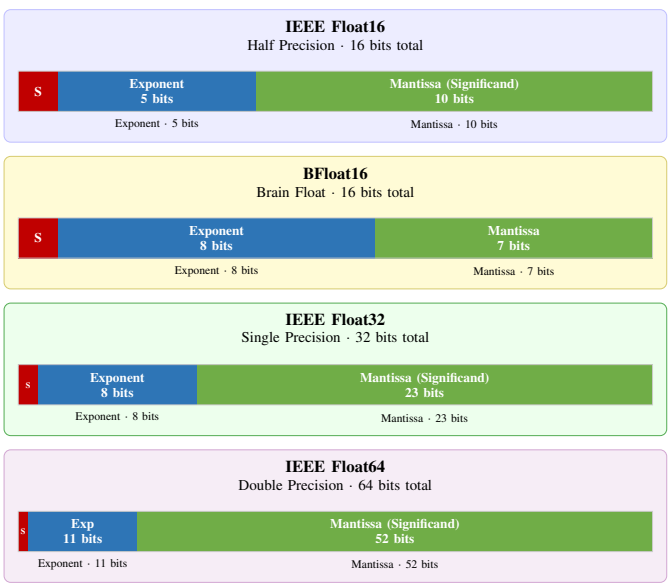

Fig. 1. Four examined floating-point representations

To compare the rate of error accumulation in each of these four formats, polynomial regression was chosen for its simplistic representation of important ML concepts such as gradient descent and loss function evaluation. Here, a final pipelined datapath will be described; numerical results will be presented to compare the accumulation of error in the aforementioned floating-point formats; hardware implementation results for the Artix 7 FPGA will be reported; and future work to improve results will be described.

II. Datapath & IP

A. Floating-Point IP

Testing multiple floating-point formats requires a significant amount of flexibility from the floating-point unit (FPU) used to evaluate the operations. In the case of this project, it is necessary that the FPU supports inputs of at least the four formats that will be tested. The most desirable characteristic of the FPU would be for these formats to be selectable by compile time Verilog parameters that determine the synthesis of the FPU. FPNew, an opensource RISCV optimized floating-point unit, was chosen because it supports exactly this kind of parameterization.

In addition to parameterizable floating-point inputs, FPNew also supports many operations including, but not limited to: addition, multiplication, division, and fused multiply add. All of these (along with subtraction, selectable as the inverse of addition) are crucial operations for the polynomial regression algorithm. The `fpnew_fma` module can synthesize hardware for any of these operations by additional parameters. The last seemingly impressive feature of FPNew is its flexible pipeline latency; parameters can be used to determine the placement of registers in the pipeline to adjust timing. This feature is not as well flushed out and will be discussed further in the synthesis report and future optimizations.

B. Data Flow & Control

Figure 2 shows the top-level datapath. Three on-chip block RAM instances form the memory system: `data_mem` holds the M training samples as packed $\{y_i, x_i\}$ words and is read-only during training; `coef_mem` holds the $N+1$ current coefficient values $\mathbf{a} = [a_0, \ldots, a_N]$ and is updated each iteration; and `error_mem` holds the M per-sample prediction errors $e_i = y_i - \hat{y}_i$ written by the forward pass and consumed by the reverse pass.

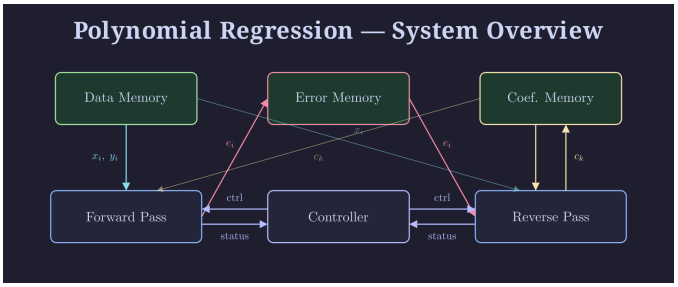

Fig. 2. Polynomial regression hardware dataflow

Forward Pass: The forward pass module processes one training sample $\{x_i, y_i\}$ per invocation. An `fp_power` state machine first computes $x_i^0, x_i^1, \ldots, x_i^N$ sequentially, outputting each power alongside its exponent index on `exp_out_valid`. The $k = 0$ term is special-cased: since $x^0 = 1$, its contribution (a_0) is loaded directly from `coef_mem` into the accumulate register, bypassing the FMA. For all higher powers $k \geq 1$, the `accumulate_coefficients` FMA computes:

$$\text{acc} \leftarrow a_k \cdot x_i^k + \text{acc} \tag{1}$$

After $N + 1$ powers are produced, `pow_done` fires. The accumulated predicted result ($\hat{y}_i$) then feeds the `error_calc` FMA as a subtraction:

$$e_i = y_i - \hat{y}_i \tag{2}$$

Because the forward pass is fully pipelined with fixed latency, two shift-register delay chains align asynchronous data with the error calculation. `y_delay` holds y_i for $L_y =$ `POW_DONE_LAT` $+ L_{\text{FMA}} + 1$ cycles so it arrives simultaneously with $\hat{y}_i$. `addr_delay` carries the sample address forward to generate the correct `error_mem` write address when `error_rdy` fires.

A one-cycle register stage between `fp_power` and the accumulation FMA further compensates for the one-cycle read latency of the synchronous block RAM storing coefficients, ensuring `coef_value` is valid when the FMA consumes it.

Reverse Pass: After all M samples are processed, the reverse pass updates each coefficient a_k using the stored errors. For each $k = 0, 1, \ldots, N$, the reverse pass streams all M samples through its own `fp_power` instance to produce x_i^k, then accumulates the gradient using a fused negative multiply-subtract (FNMSUB) instance of `fpnew_fma`:

$$\text{acc} \leftarrow -(x_i^k \cdot e_i) + \text{acc} \implies \text{error_accum} = -\sum_{i=0}^{M-1} x_i^k \, e_i \tag{3}$$

Once the last sample is accumulated, `rev_accum_rst` pulses, triggering the coefficient update FMA (also FNMSUB):

$$a_k^{(t+1)} \leftarrow -\left(\frac{2\alpha}{M}\right) \cdot \text{error_accum} + a_k^{(t)} = a_k^{(t)} + \frac{2\alpha}{M} \sum_i e_i x_i^k \tag{4}$$

This is the gradient descent step ${a_k}^{(t+1)} = {a_k}^{(t)} - \alpha \nabla_{a_k} L$ for the MSE loss, where $\nabla_{a_k} L = -\frac{2}{M} \sum_i e_i x_i^k$, which will be generalized later. The learning-rate constant $\frac{2\alpha}{M}$ is precomputed and supplied as the parameter `ALPHA_2M` to avoid a division in hardware.

Control State Machine: A nine-state FSM orchestrates the datapath:

- **INIT:** Resets all counters; waits for `go`.
- **FORWARD_PASS_START ↔ FORWARD_PASS_WAIT:** Issues one sample per handshake. `fwd_data_valid` asserts for one cycle to launch `fp_power`; the FSM then waits in FORWARD_PASS_WAIT until `fwd_pow_done` confirms the computation is complete. This repeats for all M samples.
- **REVERSE_PASS_LOOP → REVERSE_PASS_POW ↔ REVERSE_PASS_WAIT:** Outer loop increments k from 0 to N. For each k, samples are streamed in the same handshake pattern; `rev_accum_rst` pulses when all M samples are exhausted, triggering the coefficient write.
- **LOOP_BACK:** Increments the iteration counter. If `cur_pass` < `MAX_ITERATIONS`, returns to FORWARD_PASS_START for the next epoch; otherwise transitions to FINISH.
- **FINISH:** Asserts `done` until reset.

The shared `data_rd_addr` counter serves a dual purpose: it addresses `data_mem` for both the forward pass (reading x_i, y_i) and the reverse pass (reading x_i while the corresponding e_i is fetched from `error_mem` at the same address). The coefficient read address is multiplexed between the forward pass, driven by `fp_power`'s exponent counter, and the reverse pass' current coefficient index.

C. Cycles Per Epoch

The pipelined datapath processes samples in a streaming fashion; the number of clock cycles per training epoch grows linearly with polynomial degree due to the iterative x^k computation in the `fp_power` module. Table I reports measured epoch lengths across degrees. Simulation was run at 100 MHz.

TABLE I
MEASURED CLOCK CYCLES PER TRAINING EPOCH BY DEGREE ($M = 50$ SAMPLES)

Degree	Cycles/Epoch	Time/Epoch Sim @ 100MHz	Time/Epoch FP32 @ Fmax 50.9MHz
2	1,256	12.6 μs	24.7 μs
3	2,193	21.9 μs	43.1 μs
4	3,372	33.7 μs	66.2 μs
5	4,798	48.0 μs	94.3 μs
6	6,469	64.7 μs	127.1 μs

The epoch length grows roughly as $\mathcal{O}(N)$ for degree N, matching the $3 + (N-1)(L+3)$ cycle latency formula of the `fp_power` state machine (where $L = 3$ is the FMA pipeline depth). At degree 3 and the FP32 hardware Fmax

of 50.9 MHz, one full 900-iteration training run completes in approximately 38.8 ms. In comparison, for the same number of samples, epochs, and polynomial degree, a RISCV processor will execute roughly 100,000 instructions. Assuming a somewhat conservative processor operating at 1GHz and 4 cycles per instruction, the total processor execution time would take on the order of hundreds of micro-seconds. Without serious timing improvements and an improved pipeline, using this accelerator does not make sense compared to a processor. Luckily, in future work, there are plenty of opportunities for significant improvement throughout the pipeline.

III. Algorithm & Numerical Results

For each of the specified formats, polynomial regression was run on data sets from degree 2 polynomials up to degree 10 polynomials. Meaningful data was only reliably captured for polynomials up to degree 5, with extreme error results and divergence appearing at degree 6. To capture these results, multiple types of long simulation runs were executed usually with pre-calculated data points and randomized initial coefficients stored in on-chip memory.

The pre-calculated expected values for the runs shown here were calculated from polynomials with randomly generated coefficients evaluated at random inputs from the domain $X_i \in [-2, 2]$. Training used batch gradient descent with a base learning rate $\alpha = 0.01$ and $N = 200$ samples for randomized tests. The learning rate was scaled down by a factor of $|x_{\max}|^2 = 4$ for each degree above 3 to maintain gradient descent stability.

As the polynomial degree increases, the stability of gradient descent becomes exponentially harder to ensure. For this project, mean squared error was used as the loss function

$$L(\mathbf{a}) = \frac{1}{M}\|\mathbf{X}\mathbf{a} - \mathbf{y}\|^2 \tag{5}$$

where $\mathbf{a}$ is the coefficient vector, $\mathbf{X}$ is the MxN Vandermonde matrix

$$\mathbf{X} = \begin{pmatrix} 1 & x_1 & x_1^2 & \cdots & x_1^n \\ 1 & x_2 & x_2^2 & \cdots & x_2^n \\ \vdots & & & & \vdots \\ 1 & x_M & x_M^2 & \cdots & x_M^n \end{pmatrix} \tag{6}$$

representing X_i evaluated for each power in an N-degree polynomial, and $\mathbf{y}$ is the vector of expected values Y_i.

Mean squared error is an important loss function because it is twice differentiable which leads to a non-zero Hessian

$$\mathbf{H} = \frac{2}{M}\mathbf{X}^T\mathbf{X} \tag{7}$$

that gives insight to the curvature of the loss function. Importantly, the eigenbasis of $\mathbf{H}$ governs the stability condition for gradient descent because each component eigenvalue λ_k shows how $L(a_k)$ evolves. Skipping some intermediate results on the growth of the maximum eigenvalue, it can be shown that for a learning rate to converge for any N degree polynomial, it must satisfy

$$\alpha < \frac{2}{\lambda_{\max}(\mathbf{H})} \approx \frac{2n+1}{{x_{\max}}^{2n}} \tag{8}$$

where $x_{max} = 2$ is the maximum value of the input domain to the polynomial. This result leads to a key optimization for higher degree polynomials; for $N > 3$, α must scale with $1/4^{N-3}$. Finally, this yields, along with the originally defined $\alpha = 0.01$, Table II.

TABLE II
Polynomial Degree Adapted Learning Rates

Degree	4^N	Stability Threshold	Stable	Scaled LR
1	4	0.7500	Yes	0.010000
2	16	0.3125	Yes	0.010000
3	64	0.1094	Yes	0.010000
4	256	0.0352	Yes	0.002500
5	1,024	0.0107	Borderline	0.000625
6	4,096	0.0032	No	0.000156
7	16,384	0.00092	No	0.000039
8	65,536	0.00026	No	0.000009
9	262,144	0.000072	No	0.000002

Scaling the learning rate produces excellent results in terms of convergence, but still causes some issues. As the learning rate decreases, the number of epochs needed to converge on a result increases. Unfortunately, since the learning rate scales exponentially by $1/4^N$, the number of epochs scales by 4^N which leads to simulation times exploding after $N = 5$. Results are still attainable for $N = 6$, but more often than not they have an unmanageable amount of error as the algorithm has yet to converge. To analyze meaningful floating-point results, three regression scenarios will be analyzed: fixed coefficient degree 3 regression, random coefficient degree 3 regression, and cross degree random regression up to degree 5.

A. Fixed-Coefficient Baseline

As a baseline, a fixed cubic dataset was generated from the known coefficient set $[a_0, a_1, a_2, a_3] = [1.0,\ 2.0,\ -1.5,\ 0.5]$ with additive Gaussian noise ($\sigma = 0.3$). After 900 iterations, the hardware (FP32) recovered:

$$\hat{\mathbf{a}}_{\text{HW}} = [1.089,\ 1.942,\ -1.512,\ 0.482] \tag{9}$$

compared to the Python double-precision golden reference:

$$\hat{\mathbf{a}}_{\text{golden}} = [1.017,\ 1.927,\ -1.482,\ 0.484] \tag{10}$$

The hardware final MSE loss was 0.099 versus 0.083 for the golden model, a 19% increase attributable to accumulated rounding in the pipelined FMA units along with the difference in precision. Only the IEEE-754 Single Precision format was tested against the fixed cubic dataset. All four coefficients were recovered with absolute errors below 0.073, demonstrating both a successful convergence to a near-optimal solution and an initial successful result of the polynomial regression datapath.

B. Randomized Degree-3 Sweep

To characterize format-dependent accuracy across varied inputs, 200 randomized test cases were evaluated for each of the four floating-point formats at degree 3. Each test case drew random polynomial coefficients from $\mathcal{N}(0,1)$ and a fresh random dataset. Results are summarized in Table III and illustrated in Figures 3 and 4.

TABLE III
DEGREE-3 RANDOM SWEEP: 200 TESTS PER FORMAT ($\alpha = 0.01$, 900 ITERATIONS)

Format	Median HW MSE	Mean HW MSE	Mean Abs Err vs True	Mean Abs Err vs Golden
FP64	0.150	0.207	0.250	0.240
FP32	0.150	0.207	0.250	0.240
FP16	0.188	0.246	0.296	0.287
FP16ALT	0.487	0.769	0.598	0.590

The most striking result is the near-identical performance of FP64 and FP32: after 900 gradient descent iterations, the 32-bit format accumulates negligible additional rounding error relative to 64-bit double precision. FP16 (IEEE-754 half-precision) incurs a modest 19% increase in median MSE and a $\sim$20% increase in mean coefficient error, remaining broadly usable. FP16ALT (bfloat16), despite its wider dynamic range, performs significantly worse at this precision level—its reduced 7-bit mantissa produces coarser gradient estimates that accumulate into substantially larger final coefficient errors.

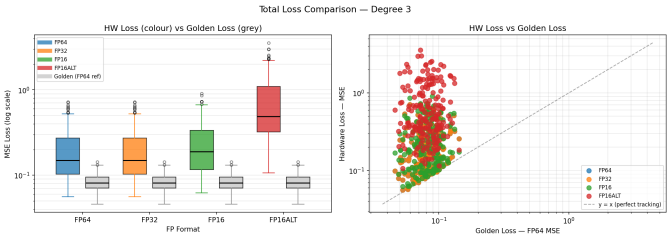

Fig. 3. Left: HW MSE vs. golden MSE per format (200 test cases, degree 3). Right: scatter of HW loss vs. golden loss.

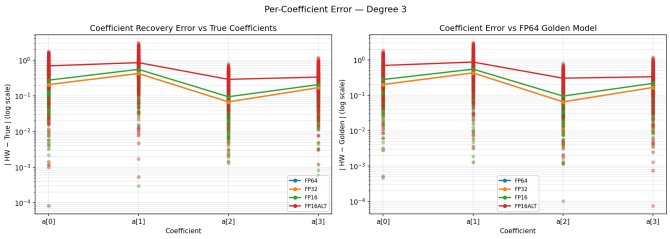

Fig. 4. Per-coefficient absolute error vs. true values (left) and vs. FP64 golden reference (right) for degree-3 random sweep. FP16ALT (red) shows consistently higher spread; FP64 and FP32 (blue/orange) are nearly coincident.

C. Cross-Degree Error Scaling

The longest running and most important set of tests was performed on a set of 200 random polynomials generated similarly to the Randomized Degree-3 Sweep with an additional random variable for the maximum degree. Because of the high number of epochs required to train a degree 6 polynomial with $\alpha = 0.000156$, these tests were allowed to run overnight to generate plenty of useful data. Figure 5 shows median MSE loss and median coefficient error across all four formats for polynomial degrees 2 through 5. Table IV reports the corresponding mean values. Even with over 1000 training epochs, output data for 6th-degree polynomials was still largely useless; of the roughly 30 6th-degree polynomials evaluated, only one converged in few enough steps. To allow for better comparison between polynomials of degree 2 through 5, degree 6 polynomials have been excluded from the results.

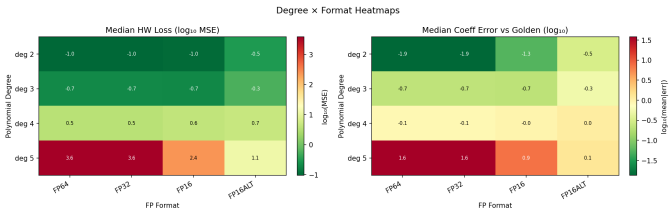

Fig. 5. Heatmap of median HW loss (left) and median coefficient error vs. golden (right) across degree and format on a $\log_{10}$ scale. Each cell reports a median over $\sim$30 random test cases.

TABLE IV
MEAN HW MSE LOSS BY DEGREE AND FORMAT

Degree	FP64	FP32	FP16	FP16ALT
2	0.098	0.098	0.104	0.311
3	0.199	0.199	0.200	0.554
4	4.41	4.41	5.09	6.17
5	9083	9083	391	25.6
6	diverged (NaN / $\gg 10^{10}$)			

Degrees 2 and 3 exhibit reasonable convergence across all formats, with losses within $10\times$ the golden reference. At degree 4 the loss jumps by two orders of magnitude; the learning rate reduction ($\alpha \to 0.0025$) is insufficient to prevent gradient magnification from the high dynamic range of x^4 over $[-2, 2]$. At degree 5, FP64 and FP32 diverge catastrophically (loss ≈ 9083), while FP16 and FP16ALT inadvertently benefit from mantissa saturation and clamping behavior that limits gradient explosion. Interestingly, even with this explosion, it seems from Figure 6 that the HW effects of error compounding in FP16ALT are log linear as opposed to the quadratic or worse effects in other formats. The inversion of lower precision outperforming higher precision illustrates that format selection is non-trivially coupled to the convergence dynamics of the gradient descent algorithm. Degree 6 and above are uniformly unstable.

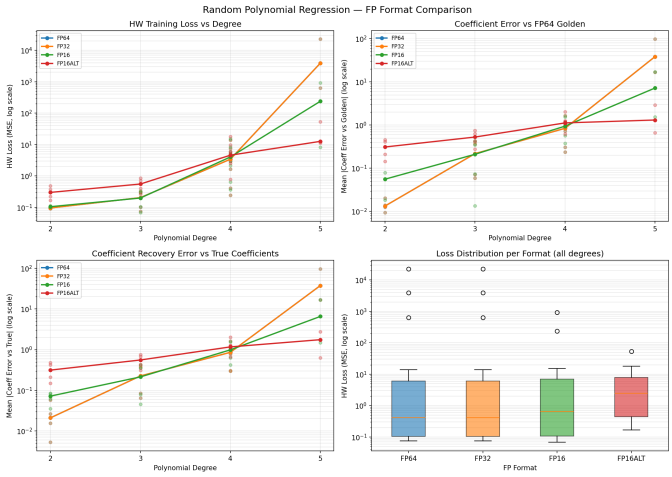

Fig. 6. Degree comparisons of MSE HW Training Loss (Top Left), Coefficient Error vs Golden (Top Right), Coefficient Error vs True Coefficients (Bottom Left), and Loss Distribution by Format (Bottom Right)

IV. Synthesis Results

All four format variants were synthesized for the Artix-7 FPGA (xc7a100tcsg324-1, speed grade -1) using Vivado 2025.2. Results are pre-place-and-route (post-synthesis estimates); actual post-P&R timing is expected to improve modestly. Figure 7 shows the synthesis summary cards for all four formats.

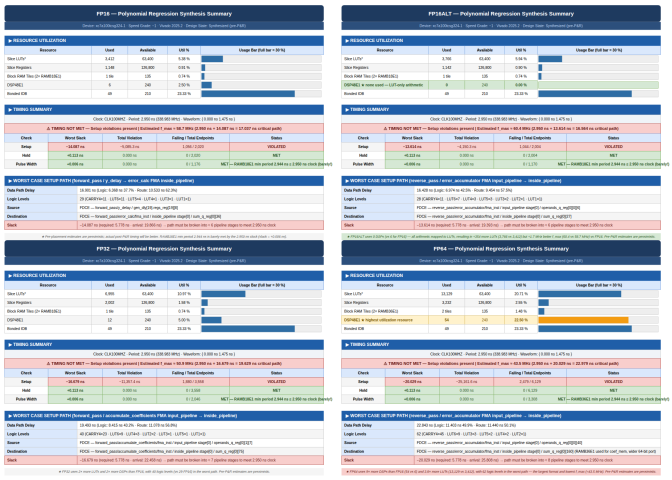

Fig. 7. Synthesis summary cards for all four formats on the Artix-7 xc7a100t. Device totals: 63,400 LUTs, 126,800 FFs, 135 BRAM tiles, 240 DSPs.

A. Resource Utilization

Table V summarises resource utilization across formats. The device constraint is the Artix-7 xc7a100t with 63,400 slice LUTs, 126,800 flip-flops, 135 BRAM tiles, and 240 DSP48E1 blocks.

TABLE V
Post-synthesis resource utilization (Artix-7 xc7a100t)

Resource	FP16	FP16ALT	FP32	FP64
Slice LUTs	3,412 (5.4%)	3,766 (5.9%)	6,955 (11.0%)	13,129 (20.7%)
Slice FFs	1,148 (0.9%)	1,142 (0.9%)	2,002 (1.6%)	3,232 (2.6%)
BRAM Tiles	1 (0.7%)	1 (0.7%)	1 (0.7%)	2 (1.5%)
DSP48E1	6 (2.5%)	0 (0.0%)	12 (5.0%)	54 (22.5%)

Several notable trends emerge. First, LUT usage scales quadratically with bit-width: going from FP16 to FP32 doubles the LUT count, while FP64 uses 3.8× more LUTs than FP16—consistent with the quadratic growth of the FMA mantissa multiplier. Second, FP16ALT (bfloat16) uses slightly more LUTs than FP16 despite its identical bit-width, because the tool could not map its non-standard exponent/mantissa split to DSP48E1 primitives—FP16ALT uses *zero* DSPs and relies entirely on LUT-based arithmetic. FP64 is the dominant consumer of DSPs at 54 of 240 (22.5%), which would leave limited room for additional FP64 compute units on the same device. At this utilization level, the Artix-7 can comfortably accommodate FP16, FP16ALT, and FP32 designs with significant headroom for peripherals or future optimizations, while FP64 saturates the DSP bank to the point where further arithmetic scaling becomes impractical.

B. Timing and Achievable Clock Frequency

All four designs violate setup timing when targeting the Nexys4-DDR's 339 MHz board clock (2.950 ns period). The critical paths are dominated by multi-level carry chains through the FMA units, with 28–62 logic levels in the worst-case setup path. Table VI reports the worst-case slack and the estimated maximum operating frequency for each format.

TABLE VI
Post-synthesis timing results (target: 2.950 ns / 339 MHz)

Format	Worst Slack	Failing Ep.	Crit. Path	Est. Fmax
FP16	−14.087 ns	1,056 / 2,020	16.90 ns (29 levels)	58.7 MHz
FP16ALT	−13.614 ns	1,044 / 2,004	16.43 ns (28 levels)	60.4 MHz
FP32	−16.679 ns	1,880 / 3,558	19.49 ns (40 levels)	50.9 MHz
FP64	−20.029 ns	2,479 / 6,129	22.84 ns (62 levels)	43.5 MHz

The FP16 and FP16ALT variants achieve the highest clock frequencies (∼59–60 MHz), with FP16ALT marginally faster despite its higher LUT count because its LUT-only arithmetic bypasses the DSP routing delay. FP32 is limited to ∼51 MHz by the wider mantissa multiplier, and FP64's critical path of 62 logic levels constrains it to ∼44 MHz. All designs' worst-case paths pass through unpipelined FMA arithmetic: the FP16/FP16ALT critical path runs from the `y_delay` register through the `error_calc` FMA input stage (forward pass), while FP32 and FP64 violations originate in the `accumulate_coefficients` and `error_accumulator` FMA input pipelines respectively.

Timing, as reported in the video documentation of this project, was slightly worse originally. The original clock target

was roughly 380MHz, but after analyzing the timing errors reported by Vivado, it was discovered that the Artix-7's block RAM primitives have a maximum read and write frequency of ∼340MHz. Adjusting the timing constraints to just below the maximum achievable RAM clocking frequency did give a little bit of extra timing improvement compared to the original results.

Bridging the gap to a practical 100 MHz operating frequency would require additional pipeline stages inserted through the FMA data paths, as noted in the Vivado timing analysis. This is achievable given the substantial remaining LUT and FF headroom.

V. Future Optimization

The results identify two primary areas for improvement: timing closure and algorithmic stability.

A. Pipelining for Timing Closure

The critical paths are 6–8 stages too deep for a 100 MHz clock and far too deep for the desired 340MHz. The FPnew FMA units expose a configurable pipeline depth (`FMA_LATENCY` parameter); in theory increasing this from the current value of 3 would retarget the critical path at the cost of proportionally increased latency. Synthesis shows that LUT and FF utilization remain well within device limits for FP16 and FP32 (<12% LUT usage), so the additional pipeline registers are feasible. FP64 would require careful management of its DSP48E1 allocation (currently 22.5%) and BRAM usage.

However, the configurable pipeline depth works counter intuitively. Rather than divying up the pipeline at multiple cut-points, there are only 3 fixed locations registers can be added: before, in the middle, and after. Registers before and after have little effect for the FMA pipeline as the inputs and outputs are already registered by the polynomial regression datapath. Including the register in the middle of the FMA pipeline was critical in getting timing to where it is now, but adding any additional registers does not disperse them throughout the arithmetic calculations to break up critical paths. Instead, it just adds more registers at the start and end of the calculation.

In other work, fully pipeline-able floating-point units for IEEE-754 Single Precision have been designed. This requires immense parameterization of every possible synthesis variable so that large combinational logic blocks (ie barrel shifters, priority encoders, and wide muxes) can be broken into smaller, registered chunks that appropriately utilize available LUTs. It is clear that FPNew needs similar configurability if it is to be able to be effectively pipelined to very high frequencies.

The extra space available to each formats hardware implementation may also allow for a pipelined rather than iterative power computation. This would increase the pipeline depth without consuming more cycles allowing a higher data throughput with no expense rather than additional resource consumption scaling linearly with the maximum polynomial degree. It is likely that for large K-polynomials this would only work up to a fixed exponent J and the remaining $K-J$ exponents would still need to be calculated iteratively.

B. Gradient Descent Stability

The degree-4+ divergence stems from a learning rate that is too large for the poorly conditioned Hessian at high degree. Adaptive learning rate schedules (e.g., Adam or RMSProp) or L2 regularisation (ridge regression) would dramatically extend the usable degree range. Ridge regression adds a $\lambda\|\mathbf{a}\|^2$ penalty that shrinks the Hessian condition number, potentially enabling stable training at degrees up to 8–9 for moderate λ. Both approaches require only minor additions to the gradient accumulation logic, which already supports general FMA chaining.

In the reverse pass, after calculating the error gradient as

$$\mathbf{E} = \mathbf{X}^T(\mathbf{X}\mathbf{a}^{(t)} - \mathbf{y}) = [\sum_{i=0}^{M} e_i {x_i}^k]_{k=0}^{N} \tag{11}$$

only one additional FMA unit is used to calculate each new ${a_k}^{(t+1)}$ according to

$$\mathbf{a}^{(t+1)} = \mathbf{a}^{(t)} - \frac{2\alpha}{M}\mathbf{E} \tag{12}$$

This extremely low hardware overhead is beneficial because it allows us to add an additional FMA unit without worrying much about space. Two extra units (one division and one FMA) are all that is necessary to regularize the data and implement ridge regression with a new coefficient update formula of

$$\mathbf{a}^{(t+1)} = \mathbf{a}^{(t)} - 2\alpha \cdot (\frac{1}{M}\mathbf{E} + \lambda\mathbf{a}^{(\mathbf{t})}) \tag{13}$$

If this improved update formula is chosen, λ will become yet another parameter that needs tuning based on polynomial degree. Theoretically, it should work well to treat λ as a hyperparameter tuned during the error streaming process. As errors build, λ can be made to increase and further penalize the increased error, and as new coefficients are calculated and error is decreased lambda can decrease. More work would need to be done to determine the exact mechanism for this scaling, but it should be possible via bit shifting for limited additional hardware resource consumption.

Lastly, in addition to an additional ridge regression term, it would also be possible to use the Hessian directly to replace gradient descent with Newton's method. This approach would be considerably more expensive in terms of hardware necessary, but would converge much more rapidly than gradient descent. Newton's method could reduce the number of required epochs considerably especially if combined with L2 regularization techniques like Ridge Regression.

VI. Links

1) https://github.com/johnabaldwin/poly_regression
2) https://github.com/ThalesGroup/fpnew